Exact Entanglement Entropy and Code Distance for Stabilizer States, Computed Without Floating-Point Arithmetic

Joel Cruz Cabrera
ORCID: 0009-0005-4048-1237

2026

Abstract

Standard quantum circuit simulation represents amplitudes as double-precision complex numbers, so quantities such as entanglement entropy are computed only approximately. We revisit two well-known facts about the stabilizer formalism — that entanglement entropy of a stabilizer state is always an integer, and that the distance of a stabilizer code is a combinatorial quantity — and implement both *exactly*, using binary linear algebra over $\mathbb{F}_2$ on the stabilizer tableau, with no floating-point step anywhere in the computation. We verify the entropy formula against Qiskit's floating-point von Neumann entropy on canonical states and on 30 random Clifford circuits (all matches exact), reproduce the area-law entanglement of the 1D cluster state, and verify the code-distance procedure against three canonical quantum error-correcting codes (Steane, the 5-qubit code, and Shor's code), recovering their published distances exactly. As an illustration of the exact toolkit in use, we report an empirical scaling law for the depth at which entanglement saturates in a specific random-circuit model. Finally, since every result above concerns exact mathematics or exact simulation of noiseless Clifford circuits, we complement it with a small, honestly-labelled real-hardware comparison: the GHZ state on 3–7 qubits, run on IBM Quantum superconducting hardware, measured against the exact noiseless prediction our formalism computes, together with a quantitative check against a naive noise model built from the device's own published calibration data — which the data systematically contradicts by 18–21 percentage points, a discrepancy we report along with our best physical explanation for it. A second, independent hardware experiment measures the cluster state's own stabilizer generators directly, and identifies a single miscalibrated physical qubit as correlated with elevated error; a follow-up control experiment, forcing the circuit onto verified low-error qubits instead, confirms the effect is causal but partial (leakage drops by 1.84 percentage points, roughly a third of the excess) rather than the whole explanation, which we report at its true, unexaggerated size. Applying the naive noise model to the control circuit itself reverses its bias (now optimistic rather than pessimistic) by an amount consistent, to order of magnitude, with ordinary decoherence over the circuit's execution time — a physical effect the naive model was never built to capture.

MSC codes: 81P45, 81P70, 94B05, 68Q12.
Keywords: stabilizer formalism, Gottesman–Knill theorem, entanglement entropy, quantum error-correcting codes, exact computation.

1 Introduction

A generic n-qubit quantum state is described by 2^n complex amplitudes, and quantities derived from it — entanglement entropy in particular — are irrational in general and are computed numerically, with floating-point rounding error. The Gottesman–Knill theorem identifies a restricted but

important class of states and circuits, those generated by the Clifford group (Hadamard, phase, and CNOT gates), for which the state admits an *exact*, discrete representation: an $n \times 2n$ binary matrix of stabilizer generators, with no continuous amplitudes at all [2, 3]. This is the mathematical content behind the classical simulability of Clifford circuits, and it underlies the entire theory of stabilizer quantum error-correcting codes.

Two quantities that are usually computed numerically for general states admit exact combinatorial descriptions in the stabilizer case:

- The *entanglement entropy* of a stabilizer state across any bipartition is an integer multiple of $\log 2$ — never irrational — computable as a rank over $\mathbb{F}_2$ of a submatrix of the stabilizer tableau [1].
- The *distance* of a stabilizer code is the minimum weight of a nontrivial logical Pauli operator, computable by an exhaustive search over the Pauli group restricted to the code's centralizer [2].

Neither fact is new; both are standard in the quantum information literature. What we report here is a from-scratch, dependency-light implementation of both quantities using pure binary linear algebra — no numerical linear algebra library, no floating-point arithmetic at any point — together with an explicit verification protocol: every claim below was checked against an independent computation (Qiskit's floating-point statevector simulator, or a published code's known parameters) before being trusted.

2 Exact entanglement entropy

Let $|\psi\rangle$ be an n-qubit stabilizer state with independent stabilizer generators $g_1, \ldots, g_n$, each written in symplectic form as a pair $(x_i, z_i) \in \mathbb{F}_2^n \times \mathbb{F}_2^n$ (so g_i acts as X on qubit q if $x_{i,q} = 1$, as Z if $z_{i,q} = 1$, as Y if both, and trivially otherwise).

Theorem 1 (Fattal, Cubitt, Yamamoto, Bravyi, Chuang, 2004 [1])**.** *For a bipartition into a region A (with complement B), the entanglement entropy of $|\psi\rangle$ across $A|B$, in units of $\log 2$, is*

$$S_A \;=\; \mathrm{rank}_{\mathbb{F}_2}(M_A) - |A|,$$

where M_A is the $n \times 2|A|$ matrix formed by taking, from each generator, only the x and z components restricted to the qubits in A.

We implement $\mathrm{rank}_{\mathbb{F}_2}$ by Gaussian elimination with XOR row operations on Python integers/booleans, avoiding NumPy or any floating-point library. The stabilizer tableau itself is read directly from Qiskit's `Clifford` object (`stab_x`, `stab_z`), which stores it as a boolean array — exact by construction, since a Clifford circuit's action on the Pauli group is itself a permutation with signs, requiring no continuous arithmetic.

2.1 Verification

We checked S_A against Qiskit's von Neumann entropy (`Statevector` + `partial_trace` + `entropy`, a floating-point computation, used here purely as an independent oracle, never as the reported value) on:

- The product state $|00\rangle$: $S_{\{0\}} = 0$.

- The Bell pair $(|00\rangle + |11\rangle)/\sqrt{2}$: $S_{\{0\}} = 1$.
- The GHZ state on n qubits, cut at a single qubit: $S_{\{0\}} = 1$ for $n = 2, 3, 5, 8$.
- The one-dimensional cluster (graph) state [8] on n qubits: $S_A = 1$ for *every* bulk cut, for $n = 4, 6, 8$ — the area-law signature expected of this state.
- Thirty random Clifford circuits ($n = 5$ qubits, 20 random gates each, uniformly random bipartition): all 30 exact values matched the floating-point reference to the precision of the float computation.

3 Exact code distance

For a stabilizer code with $n - k$ independent generators (so k logical qubits), Qiskit's `Clifford` object is not directly applicable, since it represents a full n-qubit Clifford unitary rather than a code with $k > 0$. We instead work directly with the symplectic representation.

Proposition 2. *The distance d of a stabilizer code is the minimum Hamming weight, over qubits, of a Pauli operator P satisfying:*

1. *P commutes with every stabilizer generator (symplectic inner product zero against each), and*
2. *P is not itself a product of stabilizer generators (i.e., its symplectic vector does not lie in the $\mathbb{F}_2$-span of the generators' vectors).*

We search candidates by increasing weight $w = 1, 2, \ldots$, enumerating all $\binom{n}{w}3^w$ Pauli strings of that weight, and return the first w for which a valid logical operator is found. Membership in the span is tested by reduction against a row-echelon basis of the generator matrix, again over $\mathbb{F}_2$ with no floating point.

3.1 Verification against known codes

We implemented three canonical codes independently (not imported from any coding-theory library) and computed their distance:

Code	$[[n, k, d]]$ published	d computed	Match
Steane [5] (CSS on Hamming $[7, 4, 3]$)	$[[7, 1, 3]]$	3	Yes
5-qubit perfect code [6, 7]	$[[5, 1, 3]]$	3	Yes
Shor's code [4]	$[[9, 1, 3]]$	3	Yes

These are known results; the point of the check is that our independent, from-scratch implementation reproduces them exactly, which we take as validation of the method for use on codes whose distance is not already tabulated.

4 An illustration: entanglement saturation depth

As a demonstration of the exact toolkit applied to a genuine question, we measured how quickly entanglement grows under a specific random circuit model: at each step, a gate is either a random single-qubit Clifford gate (probability 1/2) or a CNOT between a uniformly random pair of qubits, drawn from *all* $\binom{n}{2}$ pairs, not only nearest neighbors (probability 1/2). We tracked $S_A(t)$, exactly,

at every step, for a half–half bipartition, and recorded the first step at which $S_A(t)$ reaches its maximum possible value $n/2$.

Averaging over 10 random trajectories at each of $n = 10, 20, 40, 80$ qubits, the mean saturation depth t_{sat} and the ratio t_{sat}/n were:

n	mean t_{sat}	t_{sat}/n
10	144.4	14.44
20	293.9	14.69
40	553.5	13.84
80	1130.1	14.13

The ratio is stable across a factor-of-8 range in n (within ± 0.5 of 14.3), consistent with linear scaling $t_{\text{sat}}(n) \approx 14.3\, n$ for this specific circuit model. A plausible heuristic: with uniform pairing over all qubits, the probability that a given step crosses a half–half cut is $\Theta(1)$, independent of n, so the $\Theta(n)$ bits of entanglement needed to saturate should require $\Theta(n)$ steps. We report this as a numerical observation with a plausible but unproven heuristic explanation, not as a theorem; four data points with ten trajectories each do not rule out other functional forms (e.g., $n \log n$), and the specific constant is a property of this circuit model, not a universal quantity.

5 A real-hardware comparison

Everything above is either exact mathematics or exact simulation of noiseless Clifford circuits; no physical noise enters at any point. To put a number on how far actual current hardware is from the states this formalism describes, we ran the simplest circuit in the paper — the GHZ state, $|\text{GHZ}(n)\rangle = (|0^n\rangle + |1^n\rangle)/\sqrt{2}$, prepared by `ghz_state(n)` from Section 2 followed by a computational-basis measurement — on real hardware, for $n = 3, \ldots, 7$.

5.1 Exact prediction and setup

For a noiseless GHZ state, a computational-basis measurement gives $P(0^n) = P(1^n) = 1/2$ and $P(\text{anything else}) = 0$ exactly, by the definition of the state; this requires no computation beyond reading off the amplitudes. Circuits were transpiled and executed via IBM Quantum Runtime (`SamplerV2`) on `ibm_kingston`, a 156-qubit superconducting processor, at 4096 shots per circuit.

5.2 Results

Circuit	Job ID (IBM Quantum)	$P(0^n) + P(1^n)$ measured	Exact
GHZ(3), run 1	`d9kuc78ii2cc73egnv4g`	0.9441	1.0000
GHZ(3), run 2	`d9kukujhdfks73ck7g0g`	0.9412	1.0000
GHZ(3), run 3	`d9kul9bjf64c739iolog`	0.9402	1.0000
GHZ(4)	`d9kumsqbr2fc73e7tsvg`	0.9307	1.0000
GHZ(5)	`d9kuolabr2fc73e7tvf0`	0.9167	1.0000
GHZ(6)	`d9kv0tibr2fc73e7udjg`	0.8896	1.0000
GHZ(7)	`d9kv4ojjf64c739ipdgg`	0.8828	1.0000

The three GHZ(3) repetitions agree closely with one another (leakage to non-ideal outcomes: 5.59%, 5.88%, 5.98%; mean 5.82%, sample standard deviation 0.20%), indicating the hardware's error rate on this specific circuit was stable across separate job submissions, not merely noisy from run to run. Across $n = 3, \ldots, 7$, the leakage increases monotonically (5.8% $\to$ 6.9% $\to$ 8.3% $\to$

11.0% → 11.7%), consistent with the expectation that a longer CNOT chain and more measured qubits accumulate more gate and readout error — but we report this as a qualitative, seven-data-point observation, not a fitted noise model.

5.3 A quantitative check: does a naive error model predict this?

The qualitative trend above invites an obvious quantitative question: does simply multiplying IBM's own published per-gate and per-measurement error rates, for the exact physical qubits each transpiled circuit actually used, predict the observed leakage? We pulled `ibm_kingston`'s calibration data (via `backend.target`) on the same day the circuits ran, and computed, for each job,

$$P_{\text{naive}} = \prod_{\text{gate/measurement } g \text{ used}} (1 - \varepsilon_g),$$

where each ε_g is IBM's own published error rate for that specific gate on those specific physical qubits (a standard first-order circuit-fidelity estimate, not a fitted model).

n	Job ID	P_{naive} predicted	$P(0^n) + P(1^n)$ measured
3 (run 1)	`d9kuc78ii2cc73egnv4g`	0.7337	0.9441
3 (run 2)	`d9kukujhdfks73ck7g0g`	0.7337	0.9412
3 (run 3)	`d9kul9bjf64c739iolog`	0.7337	0.9402
4	`d9kumsqbr2fc73e7tsvg`	0.7286	0.9307
5	`d9kuolabr2fc73e7tvf0`	0.7187	0.9167
6	`d9kv0tibr2fc73e7udjg`	0.7104	0.8896
7	`d9kv4ojjf64c739ipdgg`	0.7025	0.8828

The naive model *underestimates* survival in all seven runs, by a systematic 18–21 percentage points — not scattered error, the same direction and a similar size every time. We checked, and ruled out, the easy explanation: the Runtime job options were submitted empty, and `SamplerV2`'s twirling and dynamical-decoupling options default to disabled, so no hidden error mitigation was applied. The more likely explanation is a known property of the underlying calibration data itself: randomized-benchmarking error rates measure a gate's total average infidelity against the ideal unitary, over *all* error types (including phase errors), not specifically the probability that a given error flips the bit ultimately read out in a computational-basis measurement. Since GHZ preparation is dominated by two-qubit gates whose errors are often phase-type, a naive bit-flip-equivalent model built from these published rates is expected to be pessimistic — which is exactly the direction of the discrepancy observed. We report this as a plausible explanation consistent with known physics, not as something we verified by a separate, independent measurement of phase- versus bit-flip-error content.

One calibration detail is worth recording on its own: physical qubit 1, used in all three GHZ(3) runs, had a published readout error of 23.4% that day — against a median of 0.9% across `ibm_kingston`'s 156 qubits (eight qubits exceeded 10%). Despite including this outlier qubit, the three GHZ(3) runs still showed only 5.6–6.0% total leakage, a further concrete illustration that a single large published error rate does not translate one-to-one into observed bit-flip probability in this kind of experiment.

5.4 A causal test: avoiding the miscalibrated qubit

The observation above is correlational: qubit 1 was present, and the leakage was elevated. To test the claim directly rather than merely observe it, we repeated the GHZ(3) circuit with the transpiler's

`initial_layout` forced onto a different, independently verified low-error chain of physical qubits (118–129–128; readout errors 0.29%, 0.34%, 0.42%, all well below the chip median), explicitly avoiding qubit 1. We confirmed the transpiler respected the requested layout before submitting.

control (no qubit 1): leakage 3.98% vs. baseline (qubit 1 included): mean 5.82%, s.d. 0.20%.

The improvement (1.84 percentage points) is real — roughly nine times the baseline's own run-to-run standard deviation, not noise — but it is *partial*, not the near-total elimination of leakage that would follow if the single qubit explained everything. Avoiding it accounts for roughly a third of the original leakage; a residual $\sim$4% leakage floor remains even on the best available qubits. We report the honest magnitude of the effect, not the more dramatic story a smaller number would have made: qubit miscalibration is a real, causally confirmed contributor, and not the whole explanation.

5.5 Where does the residual floor come from?

Applying the same naive error-rate-multiplication model of Section 5.3 to the control circuit itself (physical qubits $118, 129, 128$) gives a striking reversal: the naive model predicts 98.53% survival (1.47% leakage) — but the measured leakage was 3.98%. For the qubit-1 runs the naive model was too *pessimistic*; here, on good-quality qubits, it is too *optimistic*. The direction of the naive model's error flips with qubit quality, which is itself informative: the model has a blind spot that does not depend on which qubits are used.

That blind spot is idle-time decoherence. The naive model multiplies only *per-operation* error rates; it never accounts for the fact that a qubit must remain coherent for the circuit's entire duration, not just during the instants a gate acts on it. A rough estimate: summing the published durations of every gate and measurement in the actually-submitted control circuit gives $6.84\,\mu$s (treated as fully serial, an overestimate relative to real parallelism); dividing by the shortest T_1 among the three qubits ($T_1 = 230\,\mu$s for physical qubit 128) gives a decoherence-probability estimate of 2.97% — close to the 2.51-point gap between the naive prediction and what was measured. This is a crude estimate, not a rigorous channel model (real parallelism would shorten the effective duration; T_2, not T_1, may be the more relevant timescale for a phase-sensitive entangled state, and physical qubit 128's published $T_2 = 63.3\,\mu$s is markedly shorter than its T_1), but the order-of-magnitude agreement is consistent with a clear conclusion: the residual floor looks like ordinary decoherence during circuit execution, a physical effect the naive per-gate model was never built to capture, rather than an unexplained anomaly.

5.6 What this does and does not show

This is not a claim of quantum advantage, nor a new theorem, nor a characterisation of the device's noise in general: it is a single dated snapshot (job IDs above, executed 2026-07-29) of one specific device running one specific shallow circuit. Reproducing these exact percentages on a different day, or on different hardware, is not expected; the qubit-count trend is offered only as a plausible, physically unsurprising pattern, not a law. The control experiment is the one causal claim in this section, and even there we report a partial, bounded effect rather than overstate it as the full explanation. Likewise, the naive-model discrepancy above is a single, consistent seven-point observation with a physically plausible explanation, not a validated noise model of the device — someone with access to the device's phase-error and bit-flip-error content separately could test the explanation directly, which we did not do here. What the experiment does establish is procedural: the same circuit-construction code already verified exactly in Section 2 runs unmodified on real

hardware, and the exact, noiseless prediction this paper computes throughout is a meaningful, well-defined baseline against which real devices can be honestly measured.

6 Cluster-state stabilizers on real hardware

The GHZ comparison above tests only whether a computational-basis measurement lands on one of two ideal bitstrings. The cluster state studied in Section 4 offers a more direct test of the stabilizer structure this paper is built on: for the linear cluster state on qubits $0, \ldots, n-1$ (prepared by `cluster_state_1d(n)`), each qubit i has an associated stabilizer generator

$$K_0 = X_0 Z_1, \qquad K_i = Z_{i-1} X_i Z_{i+1} \ (0 < i < n-1), \qquad K_{n-1} = Z_{n-2} X_{n-1},$$

and, by the definition of a stabilizer state, $\langle K_i \rangle = 1$ *exactly* for every i. We measured all $n = 4$ generators on real hardware: rotate qubit i into the Z basis (an H gate) before measuring every qubit in the computational basis, then estimate $\langle K_i \rangle$ as the shot-averaged product of $(-1)^{\text{bit}}$ over the qubits in K_i's support. Before spending real hardware time, we checked this circuit-and-analysis pipeline against a noiseless exact simulation (Qiskit's `Statevector`, both the analytic expectation value and shot-sampled counts): all four generators reproduced $\langle K_i \rangle = 1.0000$ exactly, confirming the measurement logic before it was trusted on hardware.

Generator	Support	$\langle K_i \rangle$ measured	Exact
K_0	$X_0 Z_1$	0.8374	1
K_1	$Z_0 X_1 Z_2$	0.8965	1
K_2	$Z_1 X_2 Z_3$	0.9136	1
K_3	$Z_2 X_3$	0.9697	1

(`ibm_kingston`, 4096 shots per generator, 2026-07-29; job IDs in the accompanying repository.) All four circuits were transpiled to the same physical qubits used in the GHZ(3) runs of Section 5 (logical qubit i mapped to physical qubit i), including the same physical qubit 1 whose published readout error that day was 23.4%. Of the four generators, K_0, K_1, and K_2 all include physical qubit 1 in their support; K_3 is the only one that does not — and K_3 is, by a wide margin, the closest to the exact value. This is a second, independent piece of evidence for the same conclusion Section 5 reached by a different route (bit-flip statistics rather than stabilizer expectation values): a single miscalibrated physical qubit, not the number of qubits or gates in a generator, is the dominant factor in the observed deviation from the exact prediction.

7 Scope and limitations

This note does not claim any new theorem. The entropy formula is due to Fattal et al. (2004); the code-distance procedure is standard (Gottesman, 1997). What is reported here is: (a) an exact, dependency-light, floating-point-free implementation of both, (b) explicit verification against an independent computation before any result was trusted, (c) one small original observation (the saturation scaling law of Section 4), offered honestly as a numerical finding about a specific circuit model, with an unproven heuristic explanation, not a general result about entanglement growth in quantum circuits, and (d) two small, explicitly dated real-hardware comparisons (Sections 5–6) that are procedural and illustrative, not a noise-characterisation result – though the two independently point to the same qualitative conclusion, that individual qubit miscalibration dominates over circuit size in the regime we tested.

References

[1] D. Fattal, T. S. Cubitt, Y. Yamamoto, S. Bravyi, and I. L. Chuang, Entanglement in the stabilizer formalism, *arXiv:quant-ph/0406168*, 2004.

[2] D. Gottesman, Stabilizer Codes and Quantum Error Correction, Ph.D. thesis, California Institute of Technology, 1997. *arXiv:quant-ph/9705052*.

[3] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press, 2000. Chapter 10.

[4] P. W. Shor, Scheme for reducing decoherence in quantum computer memory, *Phys. Rev. A* **52**, R2493–R2496 (1995). doi:10.1103/PhysRevA.52.R2493.

[5] A. M. Steane, Error correcting codes in quantum theory, *Phys. Rev. Lett.* **77**, 793–797 (1996). doi:10.1103/PhysRevLett.77.793.

[6] R. Laflamme, C. Miquel, J. P. Paz, and W. H. Zurek, Perfect quantum error correcting code, *Phys. Rev. Lett.* **77**, 198–201 (1996). doi:10.1103/PhysRevLett.77.198.

[7] C. H. Bennett, D. P. DiVincenzo, J. A. Smolin, and W. K. Wootters, Mixed-state entanglement and quantum error correction, *Phys. Rev. A* **54**, 3824–3851 (1996). doi:10.1103/PhysRevA.54.3824.

[8] H. J. Briegel and R. Raussendorf, Persistent entanglement in arrays of interacting particles, *Phys. Rev. Lett.* **86**, 910–913 (2001). doi:10.1103/PhysRevLett.86.910.